

跨平台课程管理终端：组间协作与 API 开发规范

为了确保两周后“一键集成”成功，各模块负责人必须严格遵守以下契约。一旦定稿，严禁私自修改接口字段。

1. API 接口契约（核心：定死数据长什么样）

所有模块间的数据交换统一使用 JSON 格式。

- **字段命名规范：** 统一使用小写加下划线（snake_case），例如 `student_id`，禁止混用 `studentID` 或 `StudentId`。
- **必带基础字段：**
 - `action`: 操作类型（如 `SUBMIT`, `QUERY`, `APPROVE`）。
 - `timestamp`: Unix 时间戳。
 - `payload`: 具体业务数据。
- **状态码约定：**
 - `200`: 操作成功。
 - `400`: 客户端发来的数据格式不对（参数错误）。
 - `403`: 权限不足（如非本人提交）。
 - `500`: 后端数据库或逻辑崩溃。

2. 模块间的“握手”协议

- **模块 A（学生端） -> 模块 B（服务端）：**
 - A 组必须在上传前计算文件的 **MD5 校验码** 并随 API 发送。
 - B 组接收后必须重新计算 MD5，二者不符立即返回错误。
- **模块 B（服务端） -> 模块 C（审批端）：**
 - B 组需要提供一个 `/list/pending` 接口，返回所有待批改作业的 JSON 列表。
 - C 组根据列表渲染 TUI 界面。
- **反馈回传：**
 - C 组完成审批后，通过 API 更新数据库。B 组应触发一个“信号”，让 A 组的守护进程能拉取到最新的反馈 Markdown。

3. 路径与环境“禁令”（防翻车必看）

- **严禁硬编码路径：** 严禁在代码里写 `open("/home/zhangsan/data...")`。
 - **要求：** 所有路径必须基于 `os.path` 动态获取，或统一读取项目根目录下的 `.env` 配置文件。
- **文件操作原子性：** 模块 B 在接收大文件时，应先存在 `/tmp` 临时目录，校验 MD5 成功后再 `mv`（移动）到正式存储区，防止因传输中断产生坏文件。
- **Python 环境：** 统一使用 **Python 3.10+**，所有第三方库（如 `FastAPI`, `Requests`, `Textual`, `Watchdog`）必须记录在 `requirements.txt` 中。

4. 异常处理与日志规范（给模块 D 的验收标准）

- **统一日志：**所有模块的 `print` 必须改为日志记录。格式统一为：`[时间] [模块名] [级别] 消息内容`。
 - 示例：`[2026-05-13 14:00] [Module_A] [INFO] File saved, triggering API...`
- **超时处理：**模块 A 访问 API 时必须设置 `timeout`（建议 5 秒）。如果服务端没响应，A 组代码不能卡死，必须优雅地提示“服务器连接失败，稍后重试”。

5. 独立开发建议（Mock 机制）

- **不要等对方：**
 - **A 组和 C 组：**如果 B 组的 API 还没写好，你们可以自己用 Python 写一个极简的脚本，模拟返回一些假 JSON 数据，先让自己的界面和逻辑跑起来。
 - **B 组：**可以先用 Postman 或 `curl` 命令模拟 A 组发来的请求，测试数据库写入。

第一周结束前，**A 组和 B 组必须完成一次“空包上传”联调**（不带逻辑，只要 API 能通就行）。两周后的验收将直接在纯净的 Ubuntu 虚拟机上执行 `make install`，请各位开发者保持代码的洁净度。

项目中需要建立一个 `docs/api_spec.md` 文件，模块 B 的负责人把具体的 URL 路径（如 `POST /v1/submit`）和 JSON 样例写进去，让大家开发时对着看